

Compiler Infrastructure / Language Tooling Engineer с профессиональным опытом LLVM 22 и статического анализа C#/ .NET. Развиваю UniversalToolchain вокруг детерминированной композиции compiler pipelines; параллельно работаю с LLVM transformations, program analysis и backend/codegen validation.

2022: Компиляторы / VM · 2023: CIL + native x86-64 · 2024–2025: AST / IR / backends → toolchain · 2026 → : Профессионально: LLVM + static analysis

ПРОФЕССИОНАЛЬНЫЙ ОПЫТ

- 2026 — сейчас
- ИСП РАН — инженер по статическому анализу
- Участвую в разработке SharpChecker — платформы статического анализа C#/ .NET в ИСП РАН.
 - Работаю над задачами символьных вычислений в контексте статического анализа C#/ .NET.
- июль — август 2026
- МЦСТ — стажёр по разработке компиляторов
- Реализовал LICM-проход для LLVM 22 на C++; перенос выполняется только при выполнении условий по структуре цикла, обращениям к памяти, побочным эффектам и безопасности спекулятивного выполнения.
 - Собрал Python-обвязку для сравнения поведения программы до/после оптимизации и проверки случаев с преобразованием и без него.

ПРОЕКТЫ

- UniversalToolchain / Wist2
- Модули языка разрешают зависимости и конфликты, однозначно выбирают реализации требуемых возможностей и runtime-компонентов, получают детерминированный порядок проходов и типизированные маршруты артефактов до создания runtime.
- Текущий manifest: 1 324/1 324 теста; отдельно проверяются совпадение результатов interpreter/CIL и целостность конфигурации до исполнения.
- LLVM Interprocedural Global IV Optimization
- LLVM 22 pass локализует поддерживаемые глобальные переменные, изменяющиеся как индукционные, после межпроцедурного анализа эффектов, аффинной эволюции и отдельной проверки условий применимости; APInt моделирует арифметику фиксированной разрядности. 29 позитивных и негативных регрессионных сценариев: LLVM Verifier, структурные ожидания, идемпотентность и сравнение наблюдаемого поведения до/после преобразования.
- x86-64 Compiler Backend & Register Allocation
- Проверка SSA/CFG → анализ живости и интерференции → register allocation и spills → phi lowering → генерация кода SysV x86-64.
- Независимый verifier раскладки и differential execution: native-код сравнивается с эталонным интерпретатором; тесты покрывают ветвления, циклы, spills и parallel phi moves; отдельная публичная линия native-codegen начинается с Visk (2023) и Wist4 (2024).

КОМПЕТЕНЦИИ

- Инфраструктура компилятора
- разрешение зависимостей и конфликтов, выбор реализаций, детерминированный порядок проходов, типизированные маршруты артефактов, проверка перед запуском
- LLVM / IR
- C++23, LLVM 22/IR, CFG/SSA, анализ циклов, межпроцедурный анализ, условия применимости преобразований
- Backends / Runtime
- .NET, CIL/MSIL, native x86-64, SysV ABI, stack frames/calling conventions, register allocation, interpreter/runtime composition
- Проверка
- регрессионные тесты, LLVM Verifier, сравнение interpreter/backend, differential execution

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия

ДОСТИЖЕНИЯ

- ФКН НИУ ВШЭ — конкурс проектов с открытым кодом
- UniversalToolchain / Wist2 — один из победителей конкурса проектов с открытым кодом студентов ФКН.
- LangDev’26 — Build the Language, Then Make the Abstractions Disappear
- Доклад по архитектуре UniversalToolchain принят к выступлению на LangDev’26.
- Другие достижения
- Дважды абсолютный победитель конкурса «Юниор» НИЯУ МИФИ; призёр «Высшей пробы» по олимпиадному и промышленному программированию; диплом I степени и Главная премия Балтийского научно-инженерного конкурса.

Москва / удалённо